



BENGAL RAWR

Real-Time Academic Workload Reducer

Syllabus Clutter to Calendar

Submitted by	Ritesh Bastola
Department	Computer Information Systems
Faculty Mentor	Dr. Sarbani Banerjee
Institution	Buffalo State University, SUNY
Date Submitted	May 12 , 2026
Course	Undergraduate Research in Computing
GitHub Repo:	https://github.com/riteshb01/RAWR-public/

Department of Computer Information Systems
Buffalo State University · Spring 2026

Abstract

College students often struggle with managing academic workload because course syllabi are distributed as separate, static documents at the beginning of a semester. Important deadlines for assignments, exams, and projects are scattered across multiple PDFs, making it difficult for students to visualize their overall workload. As a result, many students discover periods of excessive academic pressure — often referred to as "impossible weeks" — only when deadlines are already approaching.

This project introduces Bengal RAWR (Real-time Academic Workload Reducer), an intelligent system designed to help students proactively manage their academic responsibilities by transforming fragmented syllabus information into a unified, actionable schedule. The system leverages Natural Language Processing (NLP) pipelines, regex-based date extraction, and a machine learning classifier to automatically extract academic events from uploaded syllabi. Events are assigned workload weights, aggregated by week, and analyzed to detect conflict periods.

The platform presents results through an interactive React dashboard featuring a GitHub-style workload heatmap, weekly bar charts, and conflict alert panels. Crucially, all processing runs locally — no external AI APIs are used — making the system privacy-preserving and fully reproducible. Results validate the core hypothesis: overlapping deadlines from different courses create hidden impossible weeks that students cannot easily foresee when reviewing individual, static documents.

Bibliography

The following resources were consulted and used during the development of Bengal RAWR:

1. Google Gemini API Documentation. (2026). Gemini API Quickstart. Google AI for Developers. <https://ai.google.dev/gemini-api/docs/quickstart>
2. Conda Documentation. (2026). Managing Environments. Anaconda Inc. <https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html>
3. Visual Studio Code. (2026). Code Editor. Microsoft. <https://code.visualstudio.com/>
4. Python Software Foundation. (2026). Python 3.13 Documentation. <https://www.python.org/>

5. Django Software Foundation. (2026). Django 6.0 Web Framework.
<https://www.djangoproject.com/>
6. PyPDF Development Team. (2026). PyPDF Documentation.
<https://pypdf.readthedocs.io/en/stable/>
7. Explosion AI. (2026). spaCy: Industrial-Strength Natural Language Processing.
<https://spacy.io/>
8. Scikit-learn Developers. (2026). scikit-learn: Machine Learning in Python. <https://scikit-learn.org/>
9. Django REST Framework. (2026). Django REST Framework Documentation.
<https://www.django-rest-framework.org/>
10. Meta Open Source. (2026). React: A JavaScript Library for Building User Interfaces.
<https://react.dev/>
11. dateparser Contributors. (2026). dateparser: Python Date Parsing Library.
<https://dateparser.readthedocs.io/>
12. pdfplumber Contributors. (2026). pdfplumber: Plumb a PDF for Detailed Information.
<https://github.com/jsvine/pdfplumber>

Project Poster

BENGAL RAWR — Real-Time Academic Workload Reducer

Ritesh Bastola · Dept. of Computer Information Systems · Faculty Mentor: Dr. Sarbani Banerjee

Research Purpose: How accurately can a local Natural Language Processing pipeline extract structured academic events from unstructured syllabus text — without relying on any external AI?

The Problem

Students have no visibility into workload collisions across courses. Multiple midterms and deadlines in one week go undetected until it's too late.

The Solution

Upload syllabi → auto-extract all deadlines → detect conflict weeks → visualize on a dashboard. 100% local, no external AI.

Pipeline



Workload Weight System

Homework 1	Assignment / Lab 2	Quiz / Presentation 2–3	Midterm / Exam 5	Final Exam 8	Conflict threshold ≥7
----------------------	------------------------------	-----------------------------------	----------------------------	------------------------	---------------------------------

Learning Objectives

- NLP fundamentals from scratch
- Information extraction pipeline design
- ML classifier training and feedback loops
- RAG (Retrieval Augmented Generation) foundations
- Local-first AI system architecture

Tech Stack: Python 3.13 · Django 6 · Django REST Framework · spaCy · scikit-learn · dateparser · pdfplumber · React 18 · SQLite · Regex

Project Description

1. Introduction

Bengal RAWR (Real-time Academic Workload Reducer) is a locally running academic planning tool that transforms fragmented course syllabi into a unified, conflict-aware schedule. At its core, the project is a research experiment in applied Natural Language Processing: can a machine reliably read a human-written syllabus and extract every deadline, exam, and assignment from it — without relying on any external AI service?

The system accepts PDF, TXT, and DOCX syllabus files, processes them through a multi-stage NLP pipeline, stores extracted events in a relational database, and presents the results through an interactive React dashboard. Workload is quantified using a weighted scoring system and aggregated by calendar week. Weeks where the cumulative workload exceeds a defined threshold are flagged as conflict periods.

This project sits at the intersection of software engineering, natural language processing, and machine learning — all executed without any dependency on third-party AI APIs such as OpenAI or Gemini. Every model, parser, and classifier runs locally on the developer's machine.

2. Background and Inspiration

The inspiration for Bengal RAWR came from a universal student experience: arriving at a week mid-semester only to discover that two midterms, a major project submission, and a quiz were all due within three days of each other. This “impossible week” phenomenon is predictable all the information exists in course syllabi from day one yet students rarely aggregate it until it is too late.

Existing tools such as Google Calendar or course management platforms like Blackboard require students to manually enter every deadline themselves. This friction means most students never build the complete picture. Bengal RAWR automates this process entirely: the student simply uploads the PDFs their professors distribute on the first day of class.

From a research perspective, the project is grounded in the field of Information Extraction (IE), a core subfield of NLP. IE systems transform unstructured natural language text into structured, machine-readable data. Syllabus documents represent a particularly challenging IE problem because they have no standard format — every professor structures them differently, uses different date formats, and describes deadlines with varying levels of specificity.

Key Research Question

How accurately can a local NLP pipeline extract structured academic events from unstructured syllabus text — without relying on any external AI?

3. Methodology

The Bengal RAWR system is built as a modular pipeline. Each stage is independent and can be tested or replaced without affecting other components. The five stages are: Document Parsing, NLP Event Extraction, Event Structuring, Conflict Detection, and Visualization.

3.1 Document Parsing

The document parser handles three file formats. For PDFs, the system uses pdfplumber to extract text page by page, with an OCR fallback via pytesseract for scanned or image-based documents. DOCX files are parsed using python-docx, which extracts text from both paragraphs and tables. Plain text files are read with multi-encoding fallback (UTF-8, Latin-1, CP1252). After extraction, text is cleaned by removing page numbers, decorative lines, headers, and excess whitespace.

3.2 NLP Event Extraction

The NLP engine uses a two-pass approach. The first pass applies compiled regular expressions to find date patterns in 8 common formats: ISO (2026-03-12), slash notation (03/12/2026), full month names (March 12, 2026), abbreviated month names (Mar 12), ordinal day-month (12th March), and relative references. The second pass uses spaCy's Named Entity Recognition (NER) model (en_core_web_sm) to identify DATE entities that the regex may have missed, providing higher recall on unusual phrasing.

Event type detection uses a keyword taxonomy with 10 categories: homework, assignment, quiz, presentation, project, midterm, exam, final, lab, and reading. Keywords are matched in priority order (final before midterm before exam) to avoid mis-classification. A context window of ± 2 lines is scanned around each keyword match to associate orphaned dates with events.

3.3 Workload Weight System

Not all academic tasks carry equal cognitive load. Bengal RAWR assigns each event type a numerical weight reflecting its relative difficulty and time commitment:

Event Type	Workload Weight
Homework / Reading	1
Assignment / Lab / Quiz	2
Presentation / Project	3
Midterm / Exam	5
Final Exam	8

Weights are summed per ISO calendar week. A week with total weight ≥ 7 is flagged as a warning; ≥ 12 is flagged as critical.

3.4 Machine Learning Classifier

A secondary classification layer uses scikit-learn's Logistic Regression model, trained on hand-crafted features extracted from each text line. Features include keyword presence (has_midterm, has_due, etc.), structural indicators (line_length, has_date_pattern, starts_with_bullet), and lexical counts. The model is bootstrapped with 27 labeled examples and improves over time by retraining on human-verified events stored in the database. Model state is persisted to disk using pickle.

3.5 Django REST API and Database

The backend is built with Django 6 and Django REST Framework. Five models store the system state: Course, SyllabusFile, Event, ConflictWeek, and AnalysisSession. All file processing runs within atomic database transactions to ensure consistency. The API exposes 12 endpoints covering upload, event listing with filters, conflict analysis, and dashboard data aggregation. SQLite is used for development with a clear migration path to PostgreSQL.

3.6 React Dashboard

The frontend is a single-page React 18 application with no UI framework dependencies. It features four views: a Dashboard with a 26-week GitHub-style workload heatmap and weekly bar chart, an Upload panel with drag-and-drop support, an Events table with type/course/date filters, and a Conflicts panel with severity-rated alert cards. All styling uses inline CSS with CSS custom properties, optimized for the IBM Plex Mono monospace aesthetic chosen to reinforce the technical identity of the project.

4. Findings and Interpretations

Testing Bengal RAWR on three real course syllabi (CIS 451, PSC 202, and MAT 311) produced meaningful and actionable results.

4.1 Event Extraction Accuracy

The regex + spaCy pipeline successfully extracted the majority of dated academic events from all three syllabi. Events with explicit date strings in standard formats (Month DD, MM/DD/YYYY, or ISO) were extracted with high precision. Events described using relative language ("before the next class," "by end of week") or ambiguous references were not extracted, which represents the primary limitation of the current approach.

4.2 Conflict Detection Results

Analysis of the combined three-course schedule identified three conflict weeks across the spring semester. Week W19 and W11 both registered warning-level loads (total weight ≥ 7), while Week W20 reached a critical load of 28 — far exceeding the threshold of 12. This critical week corresponded to the convergence of a midterm, a graded presentation, and two assignments

across different courses — precisely the kind of invisible pressure point the system is designed to surface.

4.3 Research Implications

The results validate the core hypothesis: overlapping deadlines from different courses do create hidden impossible weeks that students cannot easily foresee when reviewing individual, static documents. The system proves that shifting from reactive deadline tracking to proactive, visual time management is both technically feasible and highly effective using only locally-run NLP tools.

From the perspective of a software developer and AI/ML practitioner, the project provided hands-on experience with the unglamorous but critical work that makes production AI systems function: document parsing, text normalization, entity extraction, feature engineering, and feedback-loop model improvement. These skills directly map to roles in data engineering, NLP research, and applied ML — areas where the ability to build without relying on opaque external APIs is increasingly valued.

5. Conclusion

Bengal RAWR effectively demonstrates how NLP-driven document analysis can streamline academic planning. The system successfully reduces the manual effort required to organize course deliverables and provides a clear, unified schedule that helps mitigate academic burnout. More importantly for the developer, it provided a rigorous, end-to-end exercise in building an NLP information extraction pipeline from scratch — without outsourcing the intelligence to a third-party API.

The project reinforces a key principle in applied AI development: understanding the foundations — how machines tokenize text, recognize entities, classify intent, and surface structured data — is what distinguishes engineers who can build reliable systems from those who can only call endpoints.

Future Directions

- Google Calendar and Outlook export via ICS format
- BERT fine-tuned model for improved event extraction accuracy on non-standard syllabus formats
- Precision and recall benchmarking against a hand-labeled syllabus test set
- Multi-user authentication and per-student conflict isolation
- Docker Compose packaging for one-command deployment
- Semester-over-semester trend analysis for institutional workload research